

Distributed Systems

Topics

- 4 What is a Distributed System?
- 4 Why Distributed Systems?
- 4 Examples of Distributed Systems
- 4 Distributed System Requirements
- 4 Transparency in Distributed Systems

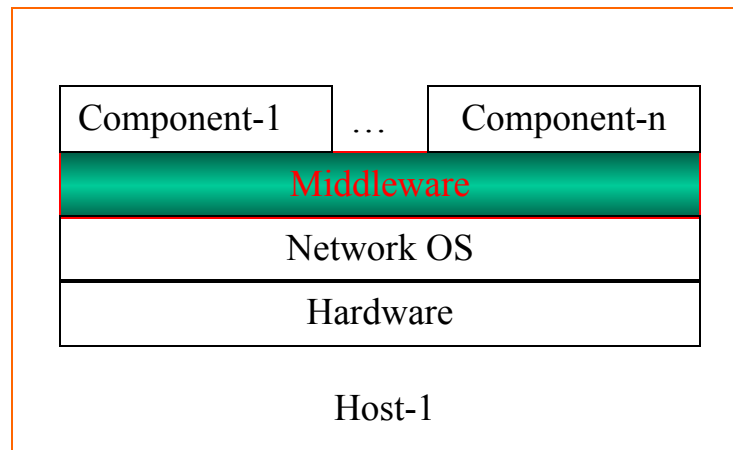
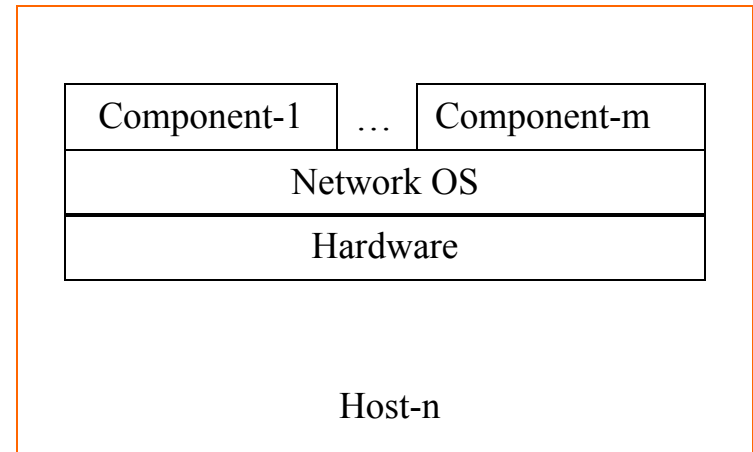
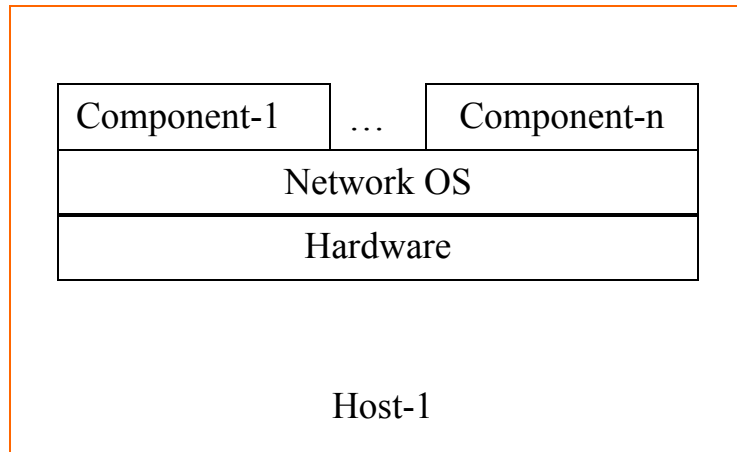
What is a Distributed System

i Distributed System Definition:

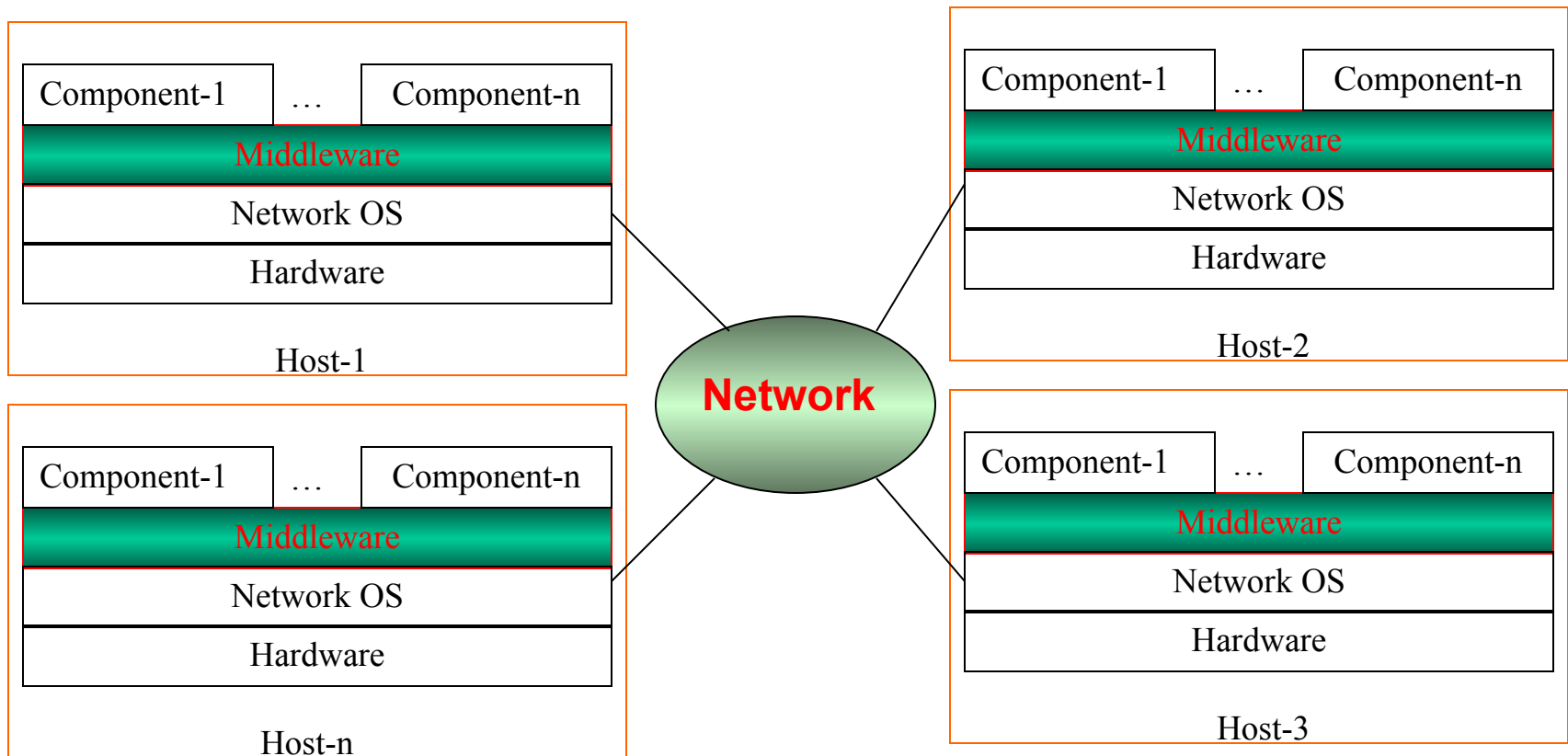
- 4 A distributed system is a collection of independent computers that appears to its users as a single coherent system.
 - 4 A distributed system is a collection of **autonomous** hosts that are connected through a computer network. Each host executes components and operates a distribution **middleware**.
 - 4 Middleware **enables** the components to **coordinate** their activities.
 - 4 Users perceive the system as a **single, integrated** computing facility.
-

What Is a Distributed System?

i Ingredients of a Distributed System



What Is a Distributed System?



Distributed Systems

i Can think of DS as:

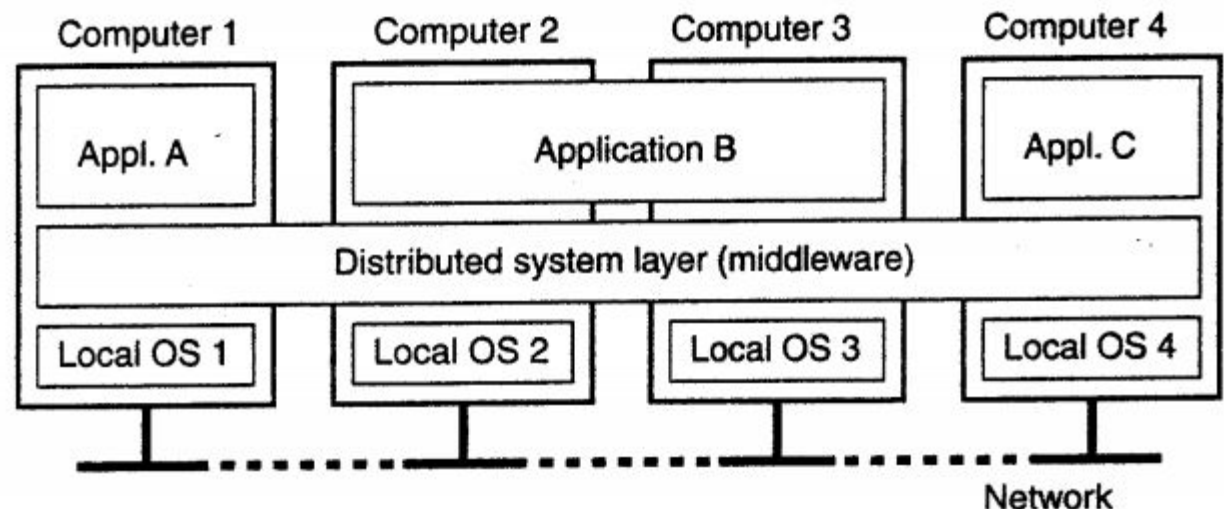
- 4** breaking down an application into individual computing agents
- 4** distributed over a network
- 4** work together on a cooperative task

i Characteristics for DC:

- 4** Differences between the various computers and the ways in which they communicate are mostly hidden from users.
- 4** users and applications can interact with a distributed system in a consistent and uniform way.
- 4** relatively easy to expand or scale

Layered Architecture of DC

- i In order to support heterogeneous computers and networks while offering a single-system view, distributed systems are often organized by means of a layer of software.
- i DC placed between a higher-level layer consisting of users and applications, and a layer consisting of operating systems and basic communication facilities.
- i Such a distributed system is sometimes called middleware. it hides the differences in hardware and operating systems from each application.



Distributed System Goals

- i Resource Accessibility**
 - i Distribution Transparency**
 - i Openness**
 - i Scalability**
-

Goals of DC

Goal 1 – Making resources accessible

- i Support user access to remote resources
(printers, data files, web pages, CPU cycles)
and the fair sharing of the resources**
 - i Economics of sharing expensive resources**
 - i Performance enhancement – due to multiple
processors; also due to ease of collaboration
and info exchange – access to remote services**
 - 4 Groupware: tools to support collaboration**
 - i Resource sharing introduces **security problems**.**
-

Goal 2 – Distribution Transparency

- i Software hides some of the details of the distribution of system resources.**
 - 4** Makes the system more user friendly.
 - i A distributed system that appears to its users & applications to be a single computer system is said to be *transparent*.**
 - 4** Users & apps should be able to access remote resources in the same way they access local resources.
 - i Transparency has several dimensions.**
-

Types of Transparency

Transparency	Description
Access	Hide differences in data representation & resource access (enables interoperability)
Location	Hide location of resource (can use resource without knowing its location)ex- http://www.prenhall.com/index.html
Migration	Hide possibility that a system may change location of resource (no effect on access)
Relocation	Hide that resource may be moved <u>during use</u>
Replication	Hide the possibility that multiple copies of the resource exist (for reliability and/or availability)
Concurrency	Hide the possibility that the resource may be shared concurrently
Failure	Hide failure and recovery of the resource.

Degrees of Transparency

- i Trade-off: transparency versus other factors**
 - i Too much emphasis on transparency may prevent the user from understanding system behavior.**
-

Goal 3 - Openness

- i An open distributed system “...offers services according to standard rules that describe the syntax and semantics of those services.” In other words, the interfaces to the system are clearly specified and freely available.**
 - i Interface Definition/Description Languages (IDL): used to describe the interfaces between software components, usually in a distributed system**
-

Open Systems Support ...

- i Interoperability: the ability of two different systems or applications to work together**
 - 4** A process that needs a service should be able to talk to any process that provides the service.
 - 4** Multiple implementations of the same service may be provided, as long as the interface is maintained
 - i Portability: an application designed to run on one distributed system can run on another system which implements the same interface.**
 - i Extensibility: Easy to add new components, features**
-

Goal 4 - Scalability

- i Dimensions that may scale:**

- 4** With respect to size

- 4** With respect to geographical distribution

- 4** With respect to the number of administrative organizations spanned

- i A system that is scalable in one or more of these dimensions often shows some loss of performance as the system scales up.**

Size Scalability

i Scalability is negatively affected when the system is based on

- 4** Centralized server: one for all users
 - 4** Centralized data: a single data base for all users
 - 4** Centralized algorithms: one site collects all information, processes it, distributes the results to all sites.
-

Geographic Scalability

- i Early distributed systems ran on LANs, relied on synchronous communication.**
 - 4 May be too slow for wide-area networks
 - 4 Wide-area communication is unreliable, point-to-point;
 - 4 Unpredictable time delays may even affect correctness

 - i LAN communication is based on broadcast.**
-

Scalability - Administrative

- i Different domains may have different policies about resource usage, management, security, etc.**
 - 4 Requires protection from malicious attacks**
 - 4 E.g. users from the other domain may be given read only access to the some files in the original domain.**
-

Scaling Techniques

- i Scalability affects performance more than anything else.**
 - i Three techniques to improve scalability:**
 - 4 Hiding communication latencies:** try to avoid waiting for responses to remote (and potentially distant) service requests as much as possible.
Distribution:
 - 4 Replication**
-

Hiding Communication Delays

- i Structure applications to use asynchronous communication (no blocking for replies)**
 - 4** While waiting for one answer, do something else; *e.g.*, create one thread to wait for the reply and let other threads continue to process or schedule another task
 - i Download part of the computation to the requesting platform to speed up processing**
 - 4** Filling in forms to access a DB: send a separate message for each field, or download form/code and submit finished version.
 - 4** *i.e.*, shorten the wait times
-

Scaling Techniques

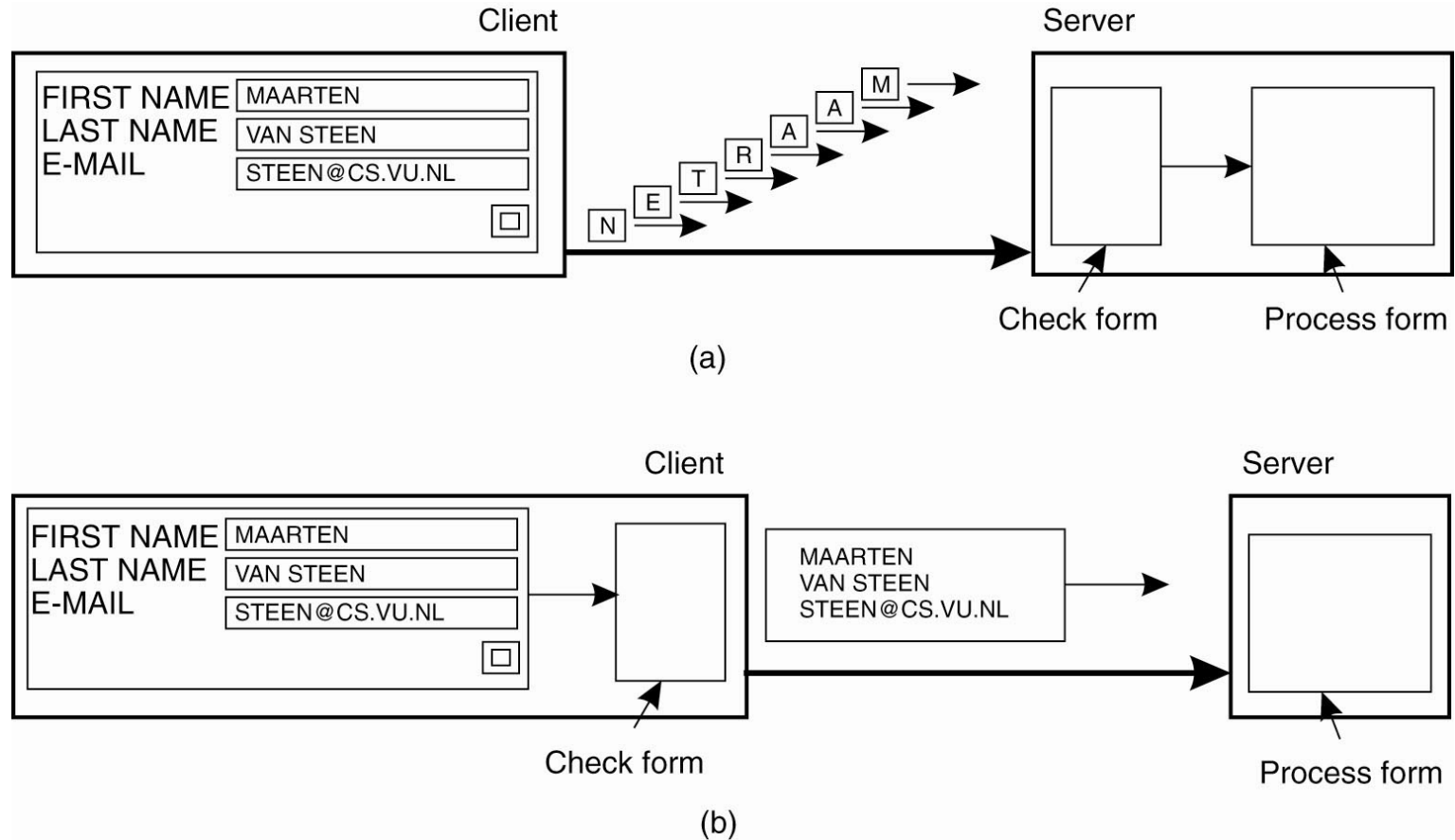


Figure 1-4. The difference between letting (a) a server or (b) a client check forms as they are being filled.

Distribution

- i Instead of one centralized service, divide into parts and distribute geographically**
 - i Each part handles one aspect of the job**
 - 4 Example: DNS namespace is organized as a tree of domains; each domain is divided into zones; names in each zone are handled by a different name server**
 - 4 WWW consists of many (millions?) of servers**
-

Scaling Techniques (2)

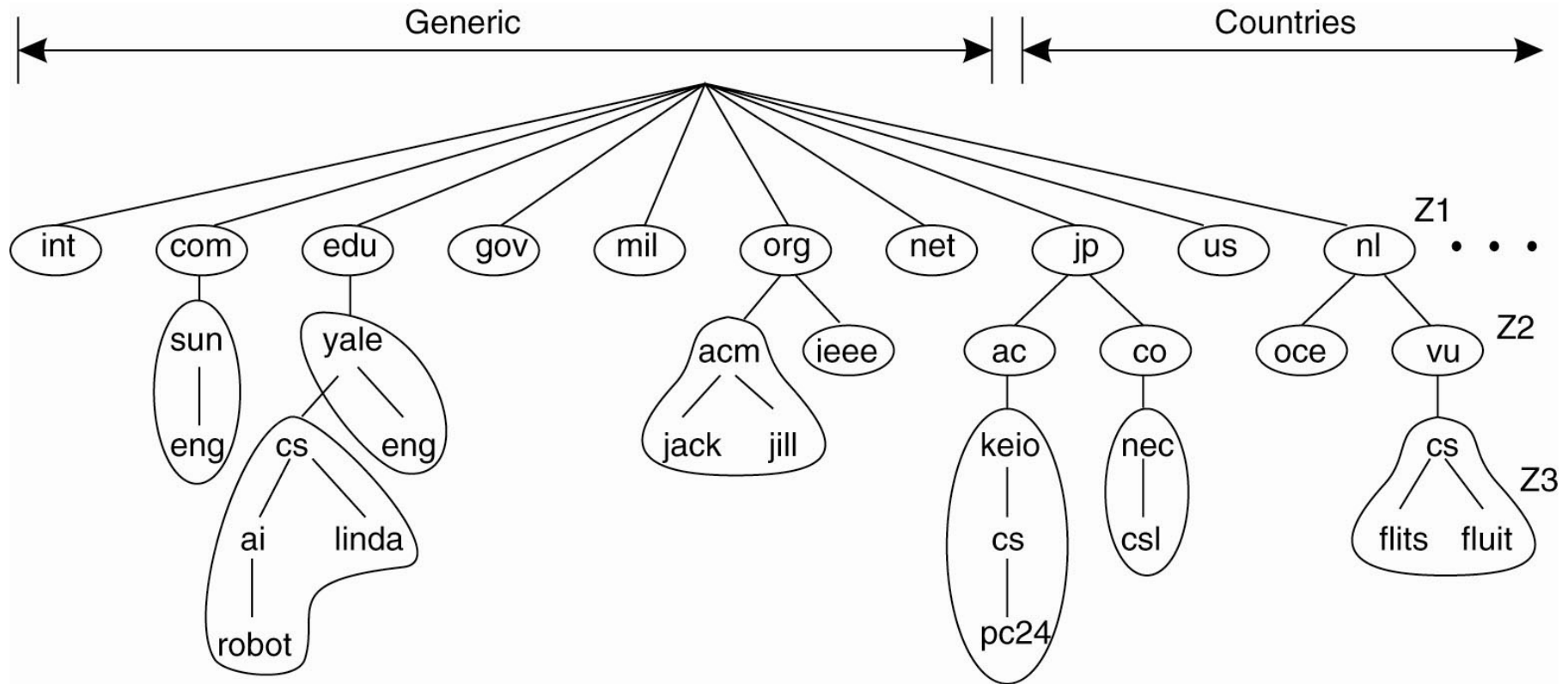


Figure 1-5. An example of dividing the DNS name space into zones.

Third Scaling Technique - Replication

- i **Replication: multiple identical copies of something**
 - 4 Replicated objects may also be distributed, but aren't necessarily.
 - i **Replication**
 - 4 Increases availability
 - 4 Improves performance through load balancing
 - 4 May avoid latency by improving proximity of resource
-

Centralized vs Distributed Systems

i Centralized Systems

- 4 Centralized systems have non-autonomous components
- 4 Centralized systems are often build using homogeneous technology
- 4 Multiple users share the resources of a centralized system at all times
- 4 Centralized systems have a single point of control and of failure

i Distributed Systems

- 4 Distributed systems have autonomous components
- 4 Distributed systems may be built using heterogeneous technology
- 4 Distributed system components may be used exclusively
- 4 Distributed systems are executed in concurrent processes
- 4 Distributed systems have multiple points of failure

Advantages and Disadvantages of Distributed Systems

i Advantages

- 4 Shareability
- 4 Expandability
- 4 Local autonomy
- 4 Improved performance
- 4 Improved reliability and availability
- 4 Potential cost reductions

i Disadvantages

- 4 Network reliance
- 4 Complexities
- 4 Security
- 4 Multiple point of failure

Examples of Distributed Systems

i Video-on-Demand

- 4 Client components are used to display videos
- 4 Clients are hosted in the homes of the customers of the service
- 4 Server components load and transmit videos
- 4 Multiple servers are needed in order to scale

Types of Distributed Systems

i Distributed Computing Systems

- 4** Clusters

- 4** Grids

i Distributed Information Systems

- 4** Transaction Processing Systems

i Distributed Embedded Systems

- 4** Home systems

- 4** Health care systems

- 4** Sensor networks

Cluster Computing

- i A collection of similar processors (PCs, workstations) running the same operating system, connected by a high-speed LAN.**
 - i Parallel computing capabilities using inexpensive PC hardware**
-

Clusters

i Master-slave paradigm

- 4 One processor is the master; allocates tasks to other processors, maintains batch queue of submitted jobs, handles interface to users
 - 4 Master has libraries to handle message-based communication or other features (the middleware).
-

Cluster Computing Systems

i **Figure 1-6. An example of a cluster computing system.**

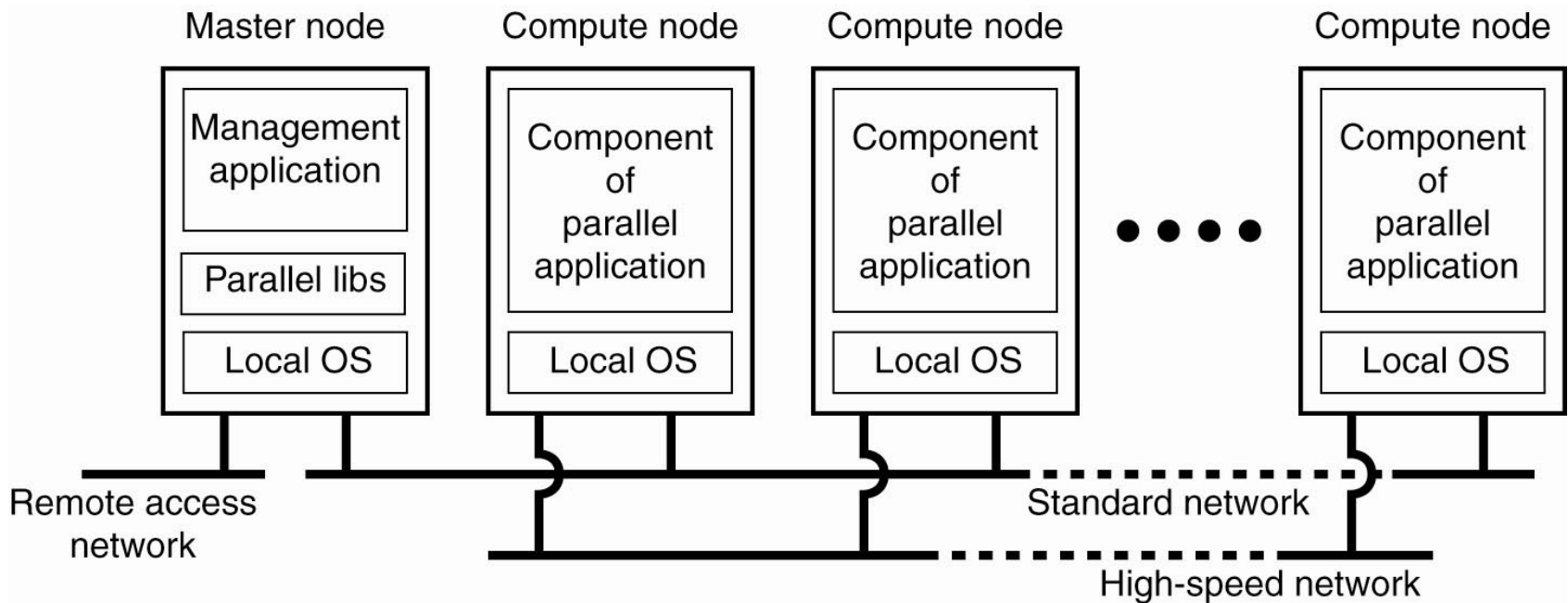


Figure 1-6. An example of a (Beowolf) cluster computing system

Grid Computing Systems

- i Highly heterogeneous with respect to hardware, software, networks, security policies, etc.**
 - i Grids support virtual organizations: a collaboration of users who pool resources (servers, storage, databases) and share them**
 - i Grid software is concerned with managing sharing across administrative domains.**
-

Grids

- i Similar to clusters but processors are more loosely coupled, tend to be heterogeneous, and are not all in a central location.**
 - i Problems are broken up into parts and distributed across multiple computers in the grid – less communication between parts than in clusters.**
-